

Mini-Capstone 2 - Subscriber Segmentation & Anomaly Detection

A graded group project. Your team applies clustering and anomaly detection to the telecom master data, then presents its findings as a slide deck.

1. What this project is

So far in class you have built a churn model - a supervised problem, where every subscriber came with a known answer (churned or not) and forecasting ARPU. This project is different. Here there is no answer key – UNSUPERVISED LEARNING. You will do two things that the data does not label for you:

- **Segmentation** - Group subscribers into segments based purely on how they behave - nobody tells you the groups exist, you have to find them.
- **Anomaly detection** - Find the small number of subscribers whose usage looks wrong - fraud and revenue leakage hidden inside 18,000 rows of monthly data.

Both techniques are new - we have not run them in class. This brief teaches you each one from scratch, with worked code you can adapt. You are not expected to already know how k-means or isolation forests work; by the end of this project, you will.

IMPORTANT Read this whole brief once before your group starts. The code here is a scaffold to adapt, not to paste blindly - the marks are in the decisions you make and the way you explain them, not in the code running.

2. Working as a group of five

This is a team submission. One notebook and one slide deck. How you divide the work is up to you, but the project splits naturally into five areas of ownership - a suggested split:

Role	Owns	Feeds into
Data & setup lead	Loading both files, feature preparation, keeping the notebook running end-to-end	Everyone depends on this
Segmentation lead	Part A - clustering, choosing k, profiling the segments	The segment table and slides 3–4
Anomaly lead	Part B - the isolation forest, tuning sensitivity, classifying flagged cases	Slides 5–6
Business analyst	Churn risk per segment, costed recommendations, the SIAM closing	Slides 4, 7
Presentation lead	Assembling the deck, the two-page summary, rehearsing the talk	The whole presentation

Every member must be able to explain the whole project, not only their part - questions in the presentation can go to anyone. The two easiest ways to lose marks as a group are a notebook that only one person understands, and a deck where the technical and business halves clearly never spoke to each other.

3. The two deliverables

Deliverable 1 - the notebook

One Jupyter notebook containing all your analysis, running top to bottom on a clean kernel without manual intervention. This is your evidence. It should be readable - short markdown notes between code cells explaining what each block does and why.

Deliverable 2 - the slide deck

A presentation your group delivers as a team. This is what actually gets read and judged by a non-technical audience. Design it yourselves - Section 8 lists the sections it must cover, but the layout, visuals and story are yours. A good deck could be followed by a campaign manager who has never opened a notebook.

4. Files

File	Use
telecom_master.csv	One row per subscriber. Segmentation, churn risk, tuning.
usage_revenue_monthly.csv	One row per subscriber per month. Anomaly detection.
Module_4_Data_Dictionary.docx	Every column explained, including when each value is recorded.

5. Part A - Subscriber segmentation

5.0 The idea, in plain language

Imagine plotting every subscriber as a dot, positioned by their behaviour - how much data they use, how long they have been with you, how much they spend. Subscribers who behave similarly sit close together; different kinds of subscriber form separate clouds of dots. Clustering is the automatic process of finding those clouds and drawing a boundary around each one. Each cloud becomes a segment.

The method you will use is called k-means. You tell it how many groups to find (that number is called k), and it works out which subscribers belong to each group by putting each one with the nearest group centre. The whole art of it is choosing a sensible k and then explaining what each group actually represents.

5.1 Choose the behaviours to cluster on

Pick columns that describe what a subscriber does, not who they are or what plan they were sold. Six is a good number. Write one line justifying each choice before you run anything.

```
import pandas as pd, numpy as np, matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

df = pd.read_csv('telecom_master.csv')

features = ['avg_monthly_gb', 'avg_voice_min', 'tenure_months',
            'arpu', 'complaints_6m', 'days_since_last_recharge']
```

WHY NOT REGION OR PLAN_TYPE? Those describe decisions the business already made, not subscriber behaviour. Leave them out of the clustering - you will use them later to describe the segments you find, which is much more useful than clustering on them.

5.2 Scale the data - the step everyone forgets

k-means measures distance between subscribers. But arpu runs into the hundreds while complaints are single digits - so without adjustment, arpu would dominate the distance and the other five behaviours would barely count. Standardising rescales every column onto the same footing.

```
X = SimpleImputer(strategy='median').fit_transform(df[features])
X = StandardScaler().fit_transform(X)
```

See it for yourself - run k-means once on the raw, unscaled columns and look at the group centres. Note which single column the groups differ on. This belongs in your notebook as evidence you understood why scaling matters.

```
raw = SimpleImputer(strategy='median').fit_transform(df[features])
km_raw = KMeans(n_clusters=4, n_init=10, random_state=42).fit(raw)
pd.DataFrame(km_raw.cluster_centers_, columns=features).round(1)
```

WHAT YOU WILL SEE The groups differ almost entirely on arpu, because it is the biggest number. That is the problem scaling fixes. Include this table in your notebook and note what it shows.

5.3 Decide how many segments - choosing k

There is no single correct k. You test a range and look at two measures. Inertia measures how tight the groups are (lower is tighter, but it always falls as you add groups). Silhouette measures how cleanly separated the groups are (higher is better, and it does not just keep rising).

```
rows = []
for k in range(2, 9):
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X)
    rows.append({'k': k,
                'inertia': km.inertia_,
                'silhouette': silhouette_score(X, km.labels_,
                                                sample_size=2000, random_state=1)})
sweep = pd.DataFrame(rows).round(3)
```

```
fig, ax1 = plt.subplots(figsize=(7, 4))
ax1.bar(sweep.k, sweep.inertia, color='#A9C4D2'); ax1.set_ylabel('Inertia')
ax2 = ax1.twinx(); ax2.plot(sweep.k, sweep.silhouette, color='#F2A03D', marker='o')
ax2.set_ylabel('Silhouette'); ax1.set_xlabel('k'); plt.show()
sweep
```

WHAT YOU WILL SEE Inertia falls smoothly with no sharp 'elbow', and silhouette stays fairly flat across several values of k. This is normal for real subscriber data. Do not invent a clean elbow that is not there - report honestly what the chart shows.

5.4 Pick your k and justify it twice

State your chosen k and defend it on two grounds: what the chart above supports, and what the business could act on. The commercial test - could the retention team genuinely run this many different campaigns? Four campaigns is realistic; twelve is not.

```
K = 4 # your choice - change and justify
km = KMeans(n_clusters=K, n_init=10, random_state=42).fit(X)
df['segment'] = km.labels_
print(df['segment'].value_counts().sort_index())
```

5.5 Profile every segment

A segment number means nothing until you describe it. Summarise each group's behaviour, then cross-tabulate against the columns you deliberately left out - this is where plan_type and device_type earn their place, as description.

```
profile = df.groupby('segment').agg(
    size      = ('customer_id', 'count'),
    mean_arpu = ('arpu', 'mean'),
    mean_tenure = ('tenure_months', 'mean'),
    mean_gb     = ('avg_monthly_gb', 'mean'),
    mean_voice  = ('avg_voice_min', 'mean'),
    mean_complaints = ('complaints_6m', 'mean'),
    churn_rate  = ('churn', 'mean'),
).round(2)
profile['share'] = (profile['size'] / len(df) * 100).round(1)
profile

print(pd.crosstab(df.segment, df.plan_type, normalize='index').round(2))
print(pd.crosstab(df.segment, df.device_type, normalize='index').round(2))
```

5.6 Name each segment and give it an action

This table is the whole point of Part A. Turn each numbered cluster into a named segment a campaign manager would recognise, with one concrete action. Produce it in exactly this shape:

Segment name	Profile	Share	Churn risk	Recommended action
e.g. Data-heavy young urban	2–3 defining behaviours	%	Low / Moderate / High	One concrete action
...				

LEARNING POINT If two segments would get the same campaign, you have too many. Merge them and say so in your write-up - recognising that is a finding, not a failure.

6. Part B - Anomaly & fraud detection

6.0 The idea, in plain language

Somewhere in the 18,000 monthly usage records are a small number of subscribers whose behaviour is wrong - fraud and billing faults. Nobody has labelled them. Your job is to find the rows that do not look like everything else.

The method is called an isolation forest. The intuition: pick a subscriber-month at random and see how few random cuts it takes to separate it from all the others. A perfectly ordinary record sits in a dense crowd and takes many cuts to isolate. A strange one sits far out on its own and gets isolated in just a few. Fewer cuts needed means more suspicious. It needs no labels, which is exactly why it suits this problem.

There are three different wrong patterns hidden in the data. You are not told which subscribers they belong to, or how many there are.

Pattern	What it looks like
Bypass / SIM-box fraud	Huge international minutes, almost no data, a low bill for all that traffic
Subscription fraud	Usage climbs, then revenue drops to zero and payments start failing
Revenue leakage	A month with real usage billed at exactly zero - a billing fault, not fraud

6.1 Load and look before modelling

```
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler

u = pd.read_csv('usage_revenue_monthly.csv')
m = pd.read_csv('telecom_master.csv')
print(u.shape, u.month.nunique(), 'months')
print(u.describe().round(2))
```

Note which columns have a long tail of large values and which have a suspicious number of exact zeros. Both are clues before any model runs.

6.2 Engineer features that expose the patterns

Raw columns alone will catch the loudest pattern and miss the subtler two. The trick is to build ratios that capture the shape of the behaviour, not just its size - for example, international minutes as a share of all calling is what separates a genuine international caller from bypass fraud.

```
u['intl_share'] = u.intl_min / (u.voice_min + u.intl_min + 1)
u['revenue_per_gb'] = u.revenue_inr / u.data_gb.clip(lower=0.01)
u['zero_revenue'] = (u.revenue_inr == 0).astype(int)
u['data_to_voice'] = u.data_gb / u.voice_min.clip(lower=1)

feat = ['data_gb', 'voice_min', 'intl_min', 'sms_count', 'revenue_inr',
        'failed_payment_count', 'intl_share', 'revenue_per_gb', 'data_to_voice']
```

6.3 Run the detector at three sensitivities

The contamination setting tells the model roughly what fraction of rows to treat as anomalies. It is not something you discover in the data - it is a decision about how many cases you are willing to investigate. Try three levels and see how many rows each flags.

```
X = StandardScaler().fit_transform(u[feat].replace([np.inf, -np.inf], 0).fillna(0))

for c in [0.005, 0.01, 0.02]:
    iso = IsolationForest(contamination=c, n_estimators=300, random_state=42).fit(X)
    u[f'flag_{c}'] = (iso.predict(X) == -1).astype(int)
    u[f'score_{c}'] = -iso.score_samples(X)
    print(f'contamination {c:<6} flagged {int(u[f"flag_{c}"].sum()):>4} rows')
```

6.4 Inspect and classify the top flags

A flag you have not looked at is not evidence. Pull the most suspicious rows, attach subscriber context, and classify each one by hand into one of the three patterns - or as an ordinary heavy user who is not actually a problem.

```
top = (u[u['flag_0.01'] == 1]
        .sort_values('score_0.01', ascending=False).head(20)
        .merge(m[['customer_id', 'plan_type', 'tenure_months', 'region']],
                on='customer_id', how='left'))
top[['customer_id', 'month', 'data_gb', 'voice_min', 'intl_min',
     'revenue_inr', 'failed_payment_count', 'plan_type']]
```

LEARNING POINT Some flagged rows will be ordinary heavy users, not fraud. Marking those correctly is part of the exercise - a detector that cannot tell a big-spending customer from a fraudster is not useful. Never call a flag a conclusion; it is a case for investigation.

6.5 Choose a sensitivity against a real budget

Assume a two-person revenue-assurance team that can investigate about forty-five cases a week. Work out which contamination level fits that capacity over the six-month data window, and state what you trade away at that level. Then put a cost on it - at, say, ₹450 to investigate one case, what does each level cost, and is the most sensitive setting worth it?

```
for c in [0.005, 0.01, 0.02]:
    n = int(u[f'flag_{c}'].sum())
    print(f'contamination {c}: {n} cases ~{n/26:.0f}/week invest. cost Rs {n*450:,}')
```

7. Part C - Bring it together

The final third is where the two analyses become a business story. Draw on the churn work from MoP 4.1 that you did cover in class.

1. State the churn risk of each segment (you already have churn_rate in your profile table) and identify the segment most worth worrying about.
2. Make a costed recommendation for that highest-risk segment. Use the cost figures from MoP 4.1 - a lost subscriber is worth far more than a retention call, roughly 20 to 1 - to justify acting on them.
3. Optionally, tune one model of your choice and show the improvement over an untuned baseline. This is credit-earning but not required; do not fake an improvement to have one.
4. Close with one clear recommendation in the Situation–Insight–Action–Value structure below.

Part	Means	Example
Situation	The business problem, in their words	We are losing subscribers and finding out too late
Insight	What your analysis found	One segment shows 3x the churn risk of the base
Action	What you want done	Route that segment to proactive retention
Value	What it is worth, costed	Catching them is worth roughly Rs X per quarter

8. The slide deck - what it must cover

Design the deck yourselves - the look, the flow and the visuals are your team to decide. It must cover these sections, but how you present them is up to you. Aim for a deck a campaign manager could follow without ever seeing your code.

- Title - project, team members, and each person's area of ownership.
- The problem - what you set out to find and why it matters to a telecom operator.
- Segmentation - how many segments, and the named segment table with actions. This is your centrepiece.
- Churn risk by segment - which segment is most at risk, shown visually.
- Anomaly detection - the three patterns, how many cases you found, and your chosen alert budget.
- A costed recommendation - your single strongest business recommendation, in the SIAV structure.
- What you would do next, and one honest limitation of your analysis.

PRESENTATION TIP The strongest decks spend most of their time on what the findings mean for the business and very little on how the algorithms work. If a slide contains the words 'k-means' or 'isolation forest', ask whether the audience needs it - usually they do not.

9. Submission

One submission per group, to the shared folder before the deadline. Name the files MiniCapstone_<groupname>.ipynb and MiniCapstone_<groupname>.pdf for the deck. Every group member's name goes on the title slide and in the notebook.